

Tecsup

Diseño y Desarrollo de Software

LABORATORIO N° 08

MVVM - ROOM

DOCENTE:

Juan José León Suiyon

CURSO:

Programación en Móviles
4to Ciclo

LABORATORIO — MVVM + Room

LavaCampus: lavandería universitaria

Dato	Detalle
Docente	Juan León S.
Tecnología	Kotlin + Jetpack Compose + Material 3 (sin XML), Room 2.6.1, DataStore, Navigation Compose
Patrón	MVVM + Repository (interface)
Duración	1 semana (trabajo guiado en clase + trabajo autónomo)
Entrega	Repositorio GitHub (ramas main y mejora-ia) + README + PROMPTS.md + capturas
Nota	20 puntos (ver rúbrica)

1. El reto

Construirás LavaCampus, la app de la lavandería de un campus universitario, con dos perfiles en el mismo dispositivo: el Estudiante arma un pedido eligiendo servicios y cantidades (kilos o prendas), ve el total y la fecha estimada de entrega, sigue el estado de su ropa y la recoge en un casillero; el Administrador recibe los pedidos, los avanza de estado (Recibido, En proceso, Listo, Entregado), asigna el casillero, cobra, administra las tarifas y consulta a sus clientes. Ambos perfiles comparten la misma base de datos Room: cuando el administrador marca un pedido como Listo, el estudiante lo ve con su casillero al instante.

> **Modo práctica (local)**
> Cada celular tiene su propia base de datos y ambos perfiles conviven en la app. Más adelante el Repository se conectará a un servicio web (Semana 9) y a la nube sin cambiar ViewModels ni pantallas; por eso es una interface. Usa solo datos ficticios (Ley 29733): nombres, códigos y correos inventados.

Qué es nuevo en este laboratorio: un pedido con varias líneas de detalle (relación uno a muchos), una transacción que guarda el pedido y todas sus líneas a la vez, un cálculo de total con precios congelados, una máquina de estados con condiciones (casillero libre, cobro previo a la entrega), fechas de entrega que saltan domingos (java.time), consultas con JOIN, SUM y GROUP BY para el panel, y un seguimiento de estado y los íconos de servicio dibujados con Canvas.

Competencias: diseñar entidades y relaciones uno a muchos en Room, escribir DAOs con Flow y suspend, usar transacciones, aplicar MVVM con Repository, implementar reglas de negocio y una máquina de estados en el ViewModel, construir una UI Material 3 fiel a un diseño y trabajar con Git de forma profesional.

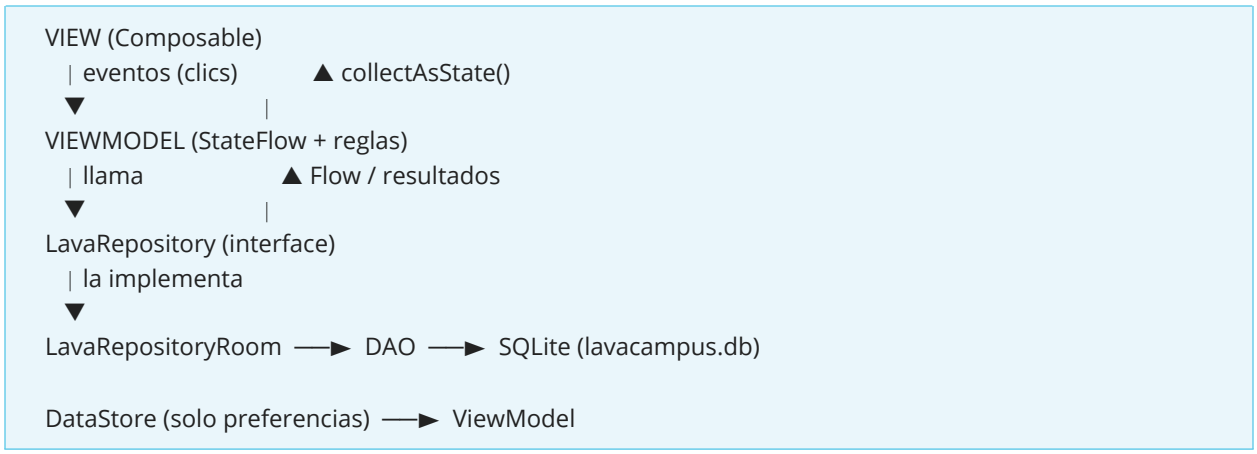
2. Cómo se trabaja: qué recibes y qué construyes

Recibirás un proyecto base que ya compila. Cada archivo del laboratorio tiene un nivel:

Nivel	Significado	Qué haces
ENTREGADO	Ya está hecho en el proyecto base	Léelo, úsalo y no lo modifiques (salvo que el paso lo indique)
GUIADO	Archivo con esqueleto y comentarios // TODO	Completas lo que falta respetando las firmas dadas
TÚ LO HACES	El archivo no existe o está vacío	Lo creas desde cero siguiendo el contrato del paso

> **Ejemplo resuelto**
> El proyecto base incluye UN vertical completo de punta a punta: Servicio (entidad), ServicioDao, sus funciones en el Repository, ServiciosViewModel y la pantalla de servicios. Estúdialo primero: el resto de la app sigue exactamente el mismo patrón.

3. Arquitectura MVVM + Repository



Capa	Conoce a	NO debe
View (ui/)	Solo a su ViewModel	Usar DAO, Repository ni reglas de negocio
ViewModel	Solo al Repository (interface)	Conocer Composables ni el DAO
Repository	Los DAOs (en la implementación)	Conocer pantallas ni ViewModels
DAO / Room	SQL y entidades	Conocer ViewModels

Regla de oro: lectura que debe actualizar la pantalla → Flow (sin suspend). Escritura o acción única → suspend. No se reemplazan, se complementan.

4. Dependencias (ya incluidas en el proyecto base)

Revísalas en build.gradle.kts del módulo app. La versión de KSP debe coincidir con la de Kotlin; si Gradle marca error al sincronizar, revisa primero esto. El proyecto usa minSdk 26 para poder emplear java.time (LocalDate, DayOfWeek) sin librerías extra.

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("org.jetbrains.kotlin.plugin.compose")
    id("com.google.devtools.ksp")        // versión = Kotlin (ej. 2.0.21-1.0.28)
}
dependencies {
    implementation(platform("androidx.compose:compose-bom:2024.09.00"))
    implementation("androidx.compose.material3:material3")
    implementation("androidx.compose.material:material-icons-extended")
    implementation("androidx.activity:activity-compose:1.9.2")
    implementation("androidx.navigation:navigation-compose:2.8.2")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.6")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.6")
    implementation("androidx.room:room-runtime:2.6.1")
    implementation("androidx.room:room-ktx:2.6.1")
    ksp("androidx.room:room-compiler:2.6.1")
    implementation("androidx.datastore:datastore-preferences:1.1.1")
}
```

5. Base de datos: 4 tablas

Tabla	Columnas (tipo)	Relación
usuarios	id (Int, PK auto) · nombre · codigo (String, ej. U202310234) · correo · clave · rol (String: CLIENTE | ADMIN)	—
servicios	id (PK auto) · nombre · descripcion · unidad (String: KG | PRENDA | UNIDAD) · precio (Double) · horasEntrega (Int) · activo (Boolean) · icono (String: wash, iron, shirt)	—
pedidos	id (PK auto) · usuarioId (FK→usuarios) · fechaRegistro (String 'aaaa-MM-dd') · fechaEntrega (String 'aaaa-MM-dd') · estado (String: RECIBIDO | EN_PROCESO | LISTO | ENTREGADO | CANCELADO) · total (Double) · casillero (String?) · pagado (Boolean) · nota · fechaCreacion (Long)	N pedidos → 1 estudiante
detalles_pedido	id (PK auto) · pedidoId (FK→pedidos) · servicioId (FK→servicios) · cantidad (Double) · precioUnitario (Double) · subtotal (Double)	N detalles → 1 pedido; N detalles → 1 servicio

¿Por qué un pedido tiene detalles en otra tabla?

Un pedido puede incluir varios servicios (lavado y planchado, por ejemplo). Guardar cada línea como una fila de detalles_pedido evita columnas repetidas y permite consultar el ingreso por servicio con un GROUP BY. Es la relación uno a muchos más común del mundo real.

¿Por qué guardar precioUnitario y subtotal en el detalle?

Es una 'foto' del momento: si mañana la lavandería sube el precio del kilo, los pedidos antiguos conservan lo que costaron y los ingresos del panel no se alteran.

Datos iniciales (se cargan solo la primera vez): 4 servicios (Lavado y secado KG S/8.00 24 h, Planchado PRENDA S/2.50 48 h, Lavado en seco PRENDA S/12.00 48 h, Edredón / frazada UNIDAD S/20.00 48 h inactivo), 3 estudiantes (ana@correo.test, luis@correo.test y marta@correo.test, clave 1234), 1 administrador (admin@lavacampus.test / admin) y unos 10 pedidos en distintos estados: algunos ENTREGADOS y pagados esta semana (para los ingresos), dos LISTOS con casillero (C-05 y C-07), algunos EN PROCESO y algunos RECIBIDOS de hoy.

6. Reglas de negocio

Estas reglas viven en los ViewModels (no en los Composables ni en los DAOs). Son lo que hace 'real' a la app:

Regla	Detalle
Visibilidad	El estudiante solo ve SUS pedidos. El administrador ve todos
Nuevo pedido: cantidades	Al menos un servicio con cantidad mayor que 0. Servicios por KG: de 1.0 a 10.0 en pasos de 0.5. Por PRENDA o UNIDAD: enteros de 1 a 30. Solo servicios ACTIVOS
Límite de pedidos	Un estudiante puede tener como máximo 3 pedidos activos (RECIBIDO, EN_PROCESO o LISTO). Al intentar un cuarto se muestra un mensaje
Total y precios	subtotal = cantidad × precio del servicio; total = suma de subtotales. Se guardan en el pedido para siempre. Estado inicial RECIBIDO y pagado = false
Fecha de entrega	Hoy + el mayor tiempo de entrega de los servicios elegidos (24 h = 1 día, 48 h = 2 días). La lavandería no atiende los domingos: si cae domingo, pasa al lunes
Registro del pedido	En UNA transacción: se guarda el pedido y todas sus líneas de detalle. Si algo falla, no queda un pedido sin detalles
Cancelar (Estudiante)	Solo si el pedido está en RECIBIDO (aún no empezó el lavado)
Cancelar (Administrador)	Puede cancelar pedidos RECIBIDO o EN_PROCESO, nunca LISTO ni ENTREGADO
Estados	RECIBIDO → EN_PROCESO → LISTO → ENTREGADO. Solo se avanza un paso a la vez y nunca se retrocede. ENTREGADO y CANCELADO son finales
Pasar a LISTO	Obliga a elegir un casillero con formato C-01 a C-30 que no esté usado por otro pedido LISTO
Pasar a ENTREGADO	Solo si el pedido está pagado. El administrador marca 'Cobrado' antes de entregar
Panel	Pedidos de hoy (sin cancelados), en proceso, por recoger (LISTO), ingresos de la semana (suma del total de pedidos pagados desde el lunes), ingresos por servicio y una alerta si hay pedidos LISTO esperando
Servicios	Un servicio que ya aparece en algún detalle no se puede eliminar (solo desactivar). Un servicio inactivo no aparece al hacer un pedido
Clientes	Lista de estudiantes con su cantidad de pedidos (sin cancelados) y lo gastado en pedidos pagados

7. Estructura de paquetes y archivos

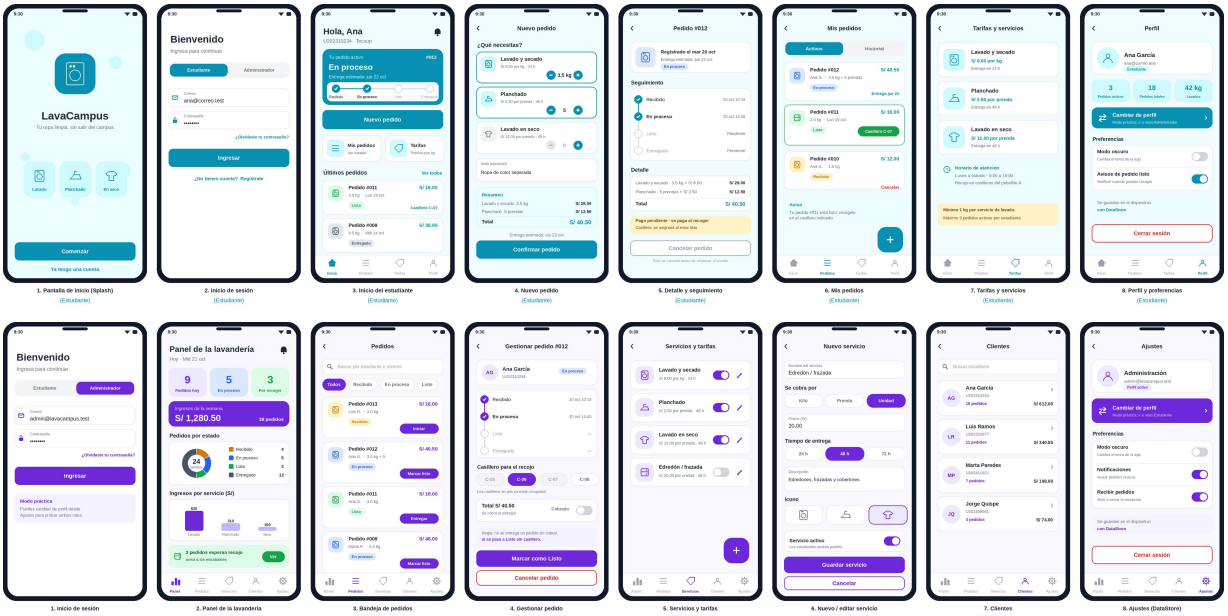
com.tecsup.lavacampus — marcado con el nivel de cada archivo: [E] entregado, [G] guiado, [A] tú lo haces.

```
com.tecsup.lavacampus
├── MainActivity.kt                [E]
├── LavaApp.kt                     [G]  crea base y repository
│
├── data
│   ├── local
│   │   ├── entity
│   │   │   ├── Servicio.kt          [E]  ← ejemplo resuelto
│   │   │   ├── Usuario.kt           [A]
│   │   │   ├── Pedido.kt            [A]
│   │   │   └── DetallePedido.kt     [A]
│   │   ├── dao
│   │   │   ├── ServicioDao.kt       [E]  ← ejemplo resuelto
│   │   │   ├── UsuarioDao.kt        [A]
│   │   │   ├── PedidoDao.kt         [A]
│   │   │   └── DetalleDao.kt        [A]
│   │   ├── relation
│   │   │   ├── DetalleConServicio.kt  [A]
│   │   │   └── PedidoConDetalle.kt    [A]
│   │   ├── LavaDatabase.kt          [G]
│   │   └── DatosIniciales.kt        [G]
│   ├── repository
│   │   ├── LavaRepository.kt        [E]  interface (el contrato)
│   │   └── LavaRepositoryRoom.kt    [G]  Servicio listo; el resto TODO
│   └── prefs
│       └── PreferenciasStore.kt     [A]
│
├── model
│   ├── Rol.kt  EstadoPedido.kt  Unidad.kt          [E]
│   ├── LineaPedido.kt  IngresoServicio.kt          [E]
│   └── ClienteResumen.kt  Casilleros.kt            [E]
│
├── viewmodel
│   ├── LavaViewModelFactory.kt      [E]
│   ├── ServiciosViewModel.kt        [G]  lista lista; guardar TODO
│   ├── SesionViewModel.kt           [A]
│   ├── InicioViewModel.kt           [A]
│   ├── NuevoPedidoViewModel.kt      [A]
│   ├── MisPedidosViewModel.kt       [A]
│   ├── DetallePedidoViewModel.kt    [A]
│   ├── PanelViewModel.kt            [A]
│   ├── BandejaViewModel.kt          [A]
│   ├── GestionPedidoViewModel.kt    [A]
│   └── ClientesViewModel.kt         [A]
│
└── ui
    ├── theme       Color.kt  Type.kt  Theme.kt          [E]
    ├── components  AppTopBar  LavaButton  LavaTextField
    │               EstadoPedidoChip  PedidoCard  StepperEstado
    │               IconoServicio  SelectorCantidad      [E]
    │   └── graficos  GraficoDona  GraficoBarras    [E] (del lab. de gráficos)
    ├── navigation  Rutas.kt [E]   NavLava.kt [G]
```

```
├── estudiante
│   ├── SplashScreen.kt          [E]
│   ├── LoginScreen.kt           [A]
│   ├── InicioScreen.kt          [A]
│   ├── NuevoPedidoScreen.kt     [A]
│   ├── DetallePedidoScreen.kt   [A]
│   ├── MisPedidosScreen.kt      [A]
│   ├── TarifasScreen.kt         [A]
│   └── PerfilScreen.kt          [A]
└── administrador
    ├── PanelScreen.kt           [A]
    ├── BandejaScreen.kt         [A]
    ├── GestionPedidoScreen.kt   [A]
    ├── ServiciosScreen.kt       [E]  ← ejemplo resuelto
    ├── FormularioServicioScreen.kt [A]
    ├── ClientesScreen.kt        [A]
    └── AjustesScreen.kt         [A]
```

8. Diseño: igual al mockup (Material 3)

Tu app debe verse como estas pantallas. Los componentes y el tema ya están entregados: úsalos, no pintes colores sueltos.



Uso	Estudiante	Administrador
Color primario	Turquesa #0891B2	Violeta #6D28D9
Contenedor	#CFFAFE	#EDE9FE
Fondo	#F0FCFF	#FAF8FF
Estado del pedido	Recibido ámbar #D97706 · En proceso azul #2563EB · Listo verde #16A34A · Entregado gris #475569 · Cancelado rojo	

Uso	Estudiante	Administrador
	#DC2626	

- LavaTheme(rol) entrega un ColorScheme según el perfil. En tus pantallas usa MaterialTheme.colorScheme.primary, nunca un color fijo.
- StepperEstado dibuja con Canvas la línea de seguimiento (Recibido, En proceso, Listo, Entregado), en versión horizontal (Inicio) y vertical (Detalle).
- IconoServicio dibuja con Canvas la lavadora, la plancha o la camisa según el campo icono del servicio.
- SelectorCantidad es el control con botones menos y más de la pantalla Nuevo pedido; recibe el valor, el paso, el mínimo y el máximo.
- Scaffold con NavigationBar: 4 ítems para el estudiante (Inicio, Pedidos, Tarifas, Perfil) y 5 para el administrador (Panel, Pedidos, Servicios, Clientes, Ajustes).
- Modo oscuro con darkColorScheme, activable desde Perfil o Ajustes y guardado en DataStore.

9. El ejemplo resuelto: estúdialo antes de empezar

Servicio.kt (entidad = tabla)

```
@Entity(tableName = "servicios")
data class Servicio(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val nombre: String,
    val descripcion: String,
    val unidad: String,        // KG, PRENDA, UNIDAD
    val precio: Double,
    val horasEntrega: Int,     // 24, 48, 72
    val activo: Boolean = true,
    val icono: String          // wash, iron, shirt
)
```

ServicioDao.kt (lectura con Flow, escritura con suspend)

```
@Dao
interface ServicioDao {
    @Query("SELECT * FROM servicios ORDER BY nombre")
    fun observarTodos(): Flow<List<Servicio>>

    @Query("SELECT * FROM servicios WHERE activo = 1 ORDER BY nombre")
    fun observarActivos(): Flow<List<Servicio>>

    @Query("SELECT * FROM servicios WHERE id = :id")
    suspend fun obtener(id: Int): Servicio?

    @Query("SELECT COUNT(*) FROM servicios")
    suspend fun contar(): Int

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertar(s: Servicio): Long
```

```
    @Insert suspend fun insertarTodos(lista: List<Servicio>)
    @Update suspend fun actualizar(s: Servicio)
    @Delete suspend fun eliminar(s: Servicio)

    @Query("UPDATE servicios SET activo = :activo WHERE id = :id")
    suspend fun cambiarActivo(id: Int, activo: Boolean)
}
```

ServiciosViewModel.kt (Flow → StateFlow con stateIn)

```
class ServiciosViewModel(private val repo: LavaRepository) : ViewModel() {
    val servicios: StateFlow<List<Servicio>> = repo.observarServicios()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())

    fun alternar(s: Servicio) {
        viewModelScope.launch { repo.cambiarActivo(s.id, !s.activo) }
    }
    // TODO (Paso 13): guardar(servicio) y eliminar(servicio)
}
```

ServiciosScreen.kt (la View solo observa y avisa)

```
@Composable
fun ServiciosScreen(vm: ServiciosViewModel, onEditar: (Int) -> Unit) {
    val servicios by vm.servicios.collectAsState()
    LazyColumn {
        items(servicios, key = { it.id }) { s ->
            ServicioItem(s, onToggle = { vm.alternar(s) }, onEditar = { onEditar(s.id) })
        }
    }
}
```

LavaRepository.kt (el contrato: tú implementas cada función en LavaRepositoryRoom)

```
interface LavaRepository {
    // Usuarios
    suspend fun login(correo: String, clave: String): Usuario?
    fun observarClientes(): Flow<List<ClienteResumen>>

    // Servicios (ejemplo resuelto)
    fun observarServicios(): Flow<List<Servicio>>
    fun observarServiciosActivos(): Flow<List<Servicio>>
    suspend fun guardarServicio(s: Servicio)
    suspend fun eliminarServicio(s: Servicio)
    suspend fun cambiarActivo(id: Int, activo: Boolean)
    suspend fun servicioTieneDetalles(id: Int): Boolean

    // Pedidos
    suspend fun crearPedido(usuarioId: Int, lineas: List<LineaPedido>,
                            fechaEntrega: String, nota: String): Long
```

```
    suspend fun contarActivos(usuarioId: Int): Int
    fun observarPedidosDeUsuario(usuarioId: Int): Flow<List<PedidoConDetalle>>
    fun observarPedidos(): Flow<List<PedidoConDetalle>>
    fun observarPedido(id: Int): Flow<PedidoConDetalle?>
    suspend fun cambiarEstado(id: Int, estado: EstadoPedido, casillero: String?)
    suspend fun marcarPagado(id: Int, pagado: Boolean)
    fun observarCasillerosOcupados(): Flow<List<String>>

    // Panel
    fun observarPedidosDeFecha(fecha: String): Flow<Int>
    fun observarCantidadPorEstado(estado: EstadoPedido): Flow<Int>
    fun observarIngresosDesde(fechaInicio: String): Flow<Double>
    fun observarIngresosPorServicio(fechaInicio: String): Flow<List<IngresoServicio>>
}
```

FASE 1 · SIN IA · Rama main

Construye LavaCampus paso a paso

Reglas de la Fase 1

No uses IA (ni Gemini ni ChatGPT). Puedes usar la documentación oficial y tus apuntes. Haz un commit por paso, con el mensaje indicado. Si tu commit mezcla varios pasos, pierdes puntos de Git.

Paso 0 · Preparar el repositorio

```
# 1. Descarga el proyecto base y ábrelo en Android Studio
# 2. Sincroniza Gradle y ejecuta: debe abrir el Splash y la pantalla de Servicios
# 3. Crea tu repositorio vacío en GitHub y conéctalo
git init
git branch -M main
git remote add origin https://github.com/TU_USUARIO/LavaCampus.git
git add .
git commit -m "chore: importa proyecto base LavaCampus"
git push -u origin main
```

COMMIT 1 — **chore: importa proyecto base LavaCampus**

La app base debe compilar y mostrar la lista de servicios.

Parte A · Datos (Room)

Paso 1 · Entidades Usuario, Pedido y DetallePedido TÚ LO HACES

Objetivo: Crear las 3 entidades que faltan, siguiendo el modelo de Servicio.kt y las tablas de la sección 5.

Archivos:

- data/local/entity/Usuario.kt
- data/local/entity/Pedido.kt
- data/local/entity/DetallePedido.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
Usuario	@Entity(tableName = "usuarios"); PK autogenerada; rol como String guardando Rol.name; codigo es el código de estudiante (vacío para el administrador)
Pedido	@Entity(tableName = "pedidos"); usuarioId con ForeignKey hacia Usuario; fechaRegistro y fechaEntrega como String; estado como String (EstadoPedido.name); total como Double; casillero String? (nulo hasta estar LISTO); pagado Boolean; fechaCreacion = System.currentTimeMillis()
DetallePedido	@Entity(tableName = "detalles_pedido"); pedidoId con ForeignKey hacia Pedido y servicioId con ForeignKey hacia Servicio; cantidad, precioUnitario y subtotal como Double

Elemento	Debe cumplir
Índices	Cada columna FK debe tener su Index. Agrega además un índice en estado de Pedido: acelera las consultas del panel y de la bandeja

> **Pista**
> Una clave foránea se declara en @Entity: foreignKeys = [ForeignKey(entity = Pedido::class, parentColumns = ["id"], childColumns = ["pedidoId"], onDelete = ForeignKey.CASCADE)], indices = [Index("pedidoId")]. Con CASCADE, al borrar un pedido se borran sus detalles.

Verifica antes del commit: las entidades compilan (aún no están registradas en la base).

COMMIT 2 **feat(data): agrega entidades Usuario, Pedido y DetallePedido**

Paso 2 · DAOs y relaciones uno a muchos TÚ LO HACES

Objetivo: Escribir las operaciones de acceso a datos y las relaciones que arman cada pedido con su estudiante y todas sus líneas de detalle.

Archivos:

- data/local/dao/UsuarioDao.kt
- data/local/dao/PedidoDao.kt
- data/local/dao/DetalleDao.kt
- data/local/relation/DetalleConServicio.kt
- data/local/relation/PedidoConDetalle.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
UsuarioDao	login(correo, clave): Usuario? (suspend) · insertarTodos(lista) · contar(): Int (suspend) · observarClientes(): Flow<List<ClienteResumen>> con JOIN a pedidos, COUNT de pedidos no cancelados y SUM del total de los pagados (0.0 si no hay), solo rol CLIENTE
PedidoDao: escritura	insertar(p): Long · actualizarEstado(id, estado, casillero) · actualizarPagado(id, pagado) · obtener(id): Pedido? (suspend)
PedidoDao: lectura	observarPorUsuario(usuarioId) · observarTodos() ordenados por id descendente · observarUno(id). Todas devuelven Flow de PedidoConDetalle con @Transaction
PedidoDao: reglas	contarActivos(usuarioId): Int (suspend, estados RECIBIDO, EN_PROCESO y LISTO) · observarCasillerosOcupados(): Flow<List<String>> con los casilleros de pedidos LISTO
PedidoDao: panel	contarPorFecha(fecha) (sin cancelados) · contarPorEstado(estado) · ingresosDesde(fechaInicio): Flow<Double> con COALESCE(SUM(total), 0.0) solo de pedidos pagados
DetalleDao	insertarTodos(lista) (suspend) · contarPorServicio(servicioId): Int (suspend) · observarIngresosPorServicio(fechaInicio): Flow<List<IngresoServicio>> con JOIN a servicios y pedidos pagados, SUM(subtotal) y GROUP BY

Elemento	Debe cumplir
DetalleConServicio	@Embedded DetallePedido + @Relation al Servicio
PedidoConDetalle	@Embedded Pedido + @Relation al Usuario + @Relation(entity = DetallePedido::class) a una List<DetalleConServicio> (relación anidada)

> **Pista**
> Una relación uno a muchos anidada se declara así: @Relation(entity = DetallePedido::class, parentColumn = "id", entityColumn = "pedidoId") val detalles: List<DetalleConServicio>. Para los ingresos por servicio: SELECT s.nombre AS nombre, SUM(d.subtotal) AS total FROM detalles_pedido d INNER JOIN servicios s ON d.servicioId = s.id INNER JOIN pedidos p ON d.pedidoId = p.id WHERE p.pagado = 1 AND p.fechaRegistro >= :fechaInicio GROUP BY s.id.

Verifica antes del commit: el proyecto compila con KSP sin errores en los DAOs.

COMMIT 3 **feat(data): agrega DAOs y relaciones PedidoConDetalle y DetalleConServicio**

Paso 3 · LavaDatabase y datos iniciales GUIADO

Objetivo: Registrar las 4 entidades y los DAOs en la base de datos y sembrar los datos de prueba la primera vez.

Archivos:

- data/local/LavaDatabase.kt
- data/local/DatosIniciales.kt
- LavaApp.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
Singleton	Una sola instancia de LavaDatabase para toda la app
DatosIniciales	suspend fun sembrar(db): si usuarioDao().contar() == 0 inserta servicios, usuarios, pedidos y sus detalles, como en la sección 5
Fechas de ejemplo	Calcula las fechas con LocalDate.now().minusDays(n).toString() para que siempre haya pedidos de hoy y de esta semana. Los pedidos ENTREGADOS van pagados; los LISTOS llevan casillero; el total de cada pedido debe coincidir con la suma de sus detalles
LavaApp	Application registrada en el AndroidManifest; al iniciar lanza sembrar(db) en una corrutina (Dispatchers.IO)

```
@Database(
    entities = [Servicio::class /* TODO: agrega las otras 3 */],
    version = 1,
    exportSchema = false
)
abstract class LavaDatabase : RoomDatabase() {
    abstract fun servicioDao(): ServicioDao
    // TODO: usuarioDao(), pedidoDao() y detalleDao()
```

```
    companion object {
        @Volatile private var INSTANCE: LavaDatabase? = null
        fun obtener(context: Context): LavaDatabase = INSTANCE ?: synchronized(this) {
            // TODO: Room.databaseBuilder(...) con nombre "lavacampus.db" (singleton)
        }
    }
}
```

> **Pista**
> Si cambias una entidad después de ejecutar la app, sube el número de version o desinstala la app del emulador; si no, Room falla porque el esquema no coincide.

Verifica antes del commit: al ejecutar, la app abre sin errores. En Android Studio: View > Tool Windows > App Inspection > Database Inspector, y revisa que existan las 4 tablas con datos y que cada pedido tenga sus detalles.

COMMIT 4 **feat(data): registra entidades y siembra datos iniciales en LavaDatabase**

Paso 4 · LavaRepositoryRoom completo GUIADO

Objetivo: Implementar en LavaRepositoryRoom todas las funciones de la interface (hoy solo las de Servicio están hechas; el resto dice TODO).

Archivos:

- data/repository/LavaRepositoryRoom.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
login y observarClientes	Delegan en UsuarioDao
crearPedido	Dentro de db.withTransaction { … }: busca cada servicio para copiar su precio, calcula subtotales y el total, inserta el Pedido (RECIBIDO, pagado = false, fechaRegistro = hoy) y luego sus DetallePedido con el id obtenido. Devuelve el id del pedido
cambiarEstado	Guarda estado.name y el casillero en la tabla. No valida reglas: las reglas viven en el ViewModel
servicioTieneDetalles	Devuelve true si detalleDao.contarPorServicio(id) es mayor que 0
observar*	Solo delegan en los DAOs; no deben usar suspend
Panel	observarIngresosPorServicio e observarIngresosDesde delegan en los DAOs con la fecha recibida

> **Pista**
> db.withTransaction es una función suspend de room-ktx: import androidx.room.withTransaction. Si el insert de un detalle falla, la transacción deshace también el insert del pedido. Piensa: ¿esta función cambia datos una sola vez o debe avisar cuando cambian? La primera es suspend y la segunda es Flow.

Verifica antes del commit: la clase compila sin ningún TODO() pendiente.

COMMIT 5 **feat(data): implementa LavaRepositoryRoom con todas las operaciones**

Paso 5 · Preferencias con DataStore TÚ LO HACES

Objetivo: Guardar en el dispositivo el perfil activo, el modo oscuro, los avisos y si la lavandería recibe pedidos.

Archivos:

- data/prefs/PreferenciasStore.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
Claves	perfilActivo (String: CLIENTE | ADMIN | vacío), usuarioId (Int), modoOscuro (Boolean), notificaciones (Boolean, por defecto true), recibirPedidos (Boolean, por defecto true)
Lectura	Un Flow por cada preferencia
Escritura	Funciones suspend: guardarSesion(rol, usuarioId), guardarModoOscuro(b), guardarNotificaciones(b), guardarRecibirPedidos(b), cerrarSesion()
Creación	Extensión val Context.dataStore by preferencesDataStore("ajustes") declarada una sola vez, a nivel de archivo

> **Pista**
> DataStore es para preferencias pequeñas (clave-valor). Los pedidos, servicios y usuarios NO van en DataStore: van en Room.

Verifica antes del commit: compila; la usarás en el Paso 6.

COMMIT 6 **feat(data): agrega PreferenciasStore con DataStore**

Parte B · Flujo del Estudiante

Paso 6 · SesionViewModel y LoginScreen TÚ LO HACES

Objetivo: Permitir el inicio de sesión con selector de rol y recordar el perfil activo.

Archivos:

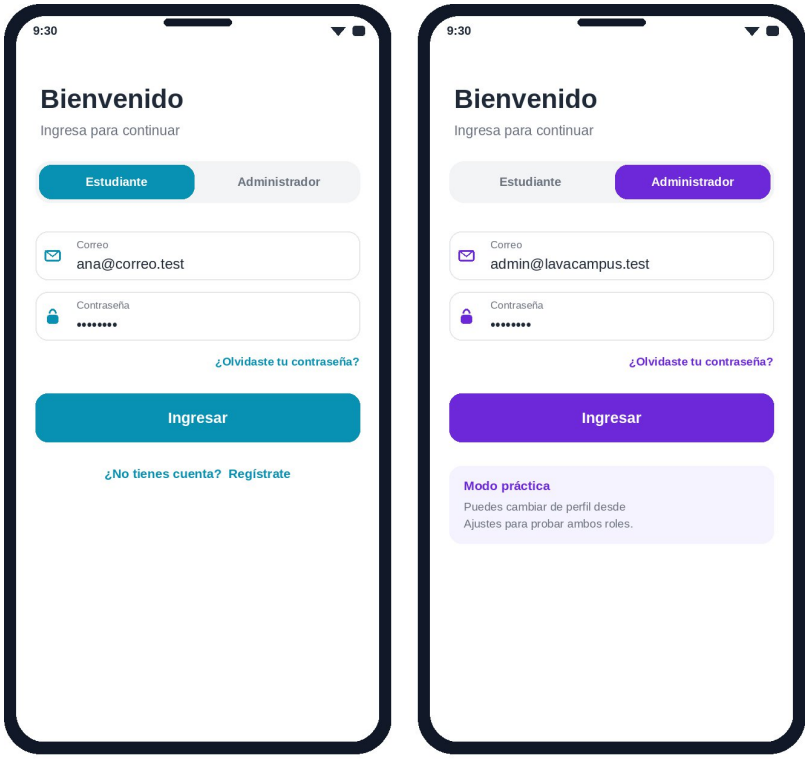
- viewmodel/SesionViewModel.kt
- ui/estudiante/LoginScreen.kt
- ui/navigation/NavLava.kt (agregar el destino)

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
SesionViewModel	Expone usuarioActual: StateFlow<Usuario?>, error: StateFlow<String?>

Elemento	Debe cumplir
	y modoOscuro. Función login(correo, clave, rolSeleccionado)
Validaciones	Campos vacíos → mensaje de error. Credenciales incorrectas → 'Correo o contraseña incorrectos'. Si el rol del usuario no coincide con la pestaña elegida → mensaje de error
Éxito	Guarda el perfil en DataStore y navega a Inicio (estudiante) o Panel (administrador)
LoginScreen	Un solo Composable usado por ambos roles: selector Estudiante / Administrador, 2 campos (LavaTextField) y botón 'Ingresar' del color del rol
Splash	Si ya hay perfil guardado, salta el login y va directo a su pantalla principal

Resultado esperado:



2. Inicio de sesión (Estudiante)

1. Inicio de sesión (Administrador)

Verifica antes del commit: entras como ana@correo.test / 1234 y como admin@lavacampus.test / admin; con datos incorrectos aparece el error.

COMMIT 7 **feat(viewmodel): agrega SesionViewModel y login con selector de rol**

Paso 7 · Inicio del estudiante TÚ LO HACES

Objetivo: Mostrar al estudiante su pedido activo con el seguimiento, accesos rápidos y sus últimos pedidos.

Archivos:

- viewmodel/InicioViewModel.kt
- ui/estudiante/InicioScreen.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
InicioViewModel	Recibe el estudiante actual y expone pedidoActivo: StateFlow<PedidoConDetalle?> y recientes: StateFlow<List<PedidoConDetalle>>. El pedido activo es el más reciente en estado RECIBIDO, EN_PROCESO o LISTO (se calcula en el ViewModel a partir de observarPedidosDeUsuario). Los recientes son los 2 últimos
InicioScreen	Saludo con el nombre y el código, tarjeta turquesa del pedido activo con StepperEstado horizontal y fecha estimada (o un mensaje 'No tienes pedidos activos'), botón 'Nuevo pedido', accesos a Mis pedidos y Tarifas, y la lista de últimos pedidos con PedidoCard
Pedido LISTO	Si el pedido está LISTO, la tarjeta muestra 'Recógelo en el casillero C-xx'
Navegación	'Nuevo pedido' abre el Paso 8; tocar un pedido abre su detalle (Paso 9)

> **Pista**
> StateFlow<PedidoConDetalle?> se obtiene con .map { lista -> lista.firstOrNull { ... } }.stateIn(...). El orden de EstadoPedido ayuda: usa EstadoPedido.valueOf(p.pedido.estado).

Resultado esperado:



3. Inicio del estudiante
(Estudiante)

Verifica antes del commit: ana@correo.test ve su pedido activo con el seguimiento y sus 2 últimos pedidos.

COMMIT 8 **feat(ui): agrega inicio del estudiante con seguimiento del pedido**

Paso 8 · Nuevo pedido TÚ LO HACES

Objetivo: Permitir al estudiante elegir servicios y cantidades, ver el total y la fecha de entrega, y confirmar el pedido.

Archivos:

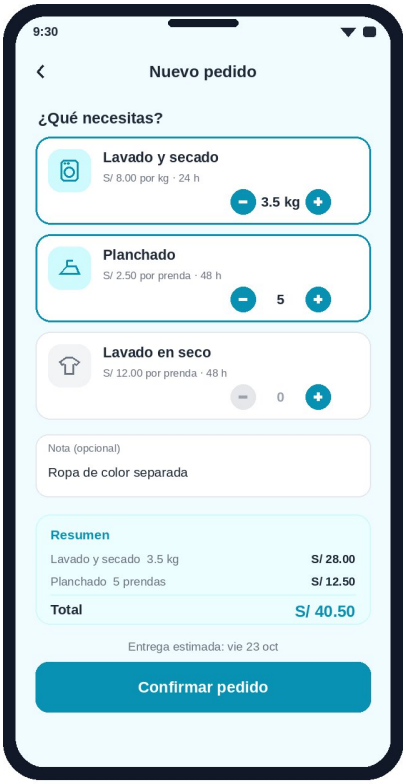
- viewmodel/NuevoPedidoViewModel.kt
- ui/estudiante/NuevoPedidoScreen.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
NuevoPedidoViewModel	Estados: servicios activos, cantidades por servicio (Map<Int, Double>), nota, total y fechaEntrega calculados. Funciones cambiarCantidad(servicioId, delta) y confirmar()
Cantidades	KG: pasos de 0.5, mínimo 1.0 al activar y máximo 10.0. PRENDA y UNIDAD: pasos de 1, máximo 30. Con cantidad 0 el servicio queda apagado
Total	Se recalcula en cada cambio: suma de cantidad × precio. Se muestra en el resumen con una línea por servicio elegido
Fecha de entrega	Hoy + el mayor horasEntrega de los servicios elegidos (24 h = 1 día). Si cae domingo, pasa al lunes (DayOfWeek.SUNDAY)
Límite	Al confirmar, si contarActivos(usuario) ya es 3, muestra 'Ya tienes 3 pedidos activos' y no crea el pedido
Botón 'Confirmar pedido'	Habilitado solo si hay al menos un servicio con cantidad mayor que 0. Al confirmar llama a repo.crearPedido y vuelve a Mis pedidos

> **Pista**
> Para saltar el domingo: var f = LocalDate.now().plusDays(dias); if (f.dayOfWeek == DayOfWeek.SUNDAY) f = f.plusDays(1). Para ver los pasos de cantidad, SelectorCantidad ya recibe paso, mínimo y máximo.

Resultado esperado:



4. Nuevo pedido
(Estudiante)

Verifica antes del commit: con 3.5 kg de lavado y 5 prendas de planchado el total es S/ 40.50; sin servicios el botón está apagado; con 3 pedidos activos no deja crear el cuarto.

COMMIT 9 **feat(ui): agrega nuevo pedido con total y fecha de entrega**

Paso 9 · Detalle del pedido y cancelación TÚ LO HACES

Objetivo: Mostrar el seguimiento completo de un pedido y permitir cancelarlo cuando la regla lo permita.

Archivos:

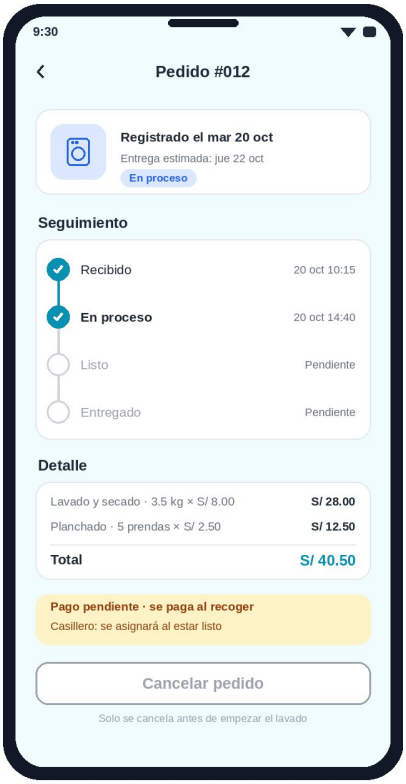
- viewmodel/DetallePedidoViewModel.kt
- ui/estudiante/DetallePedidoScreen.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
DetallePedidoViewModel	pedido: StateFlow<PedidoConDetalle?> (observarPedido) y función cancelar(). Calcula puedeCancelar = estado RECIBIDO
Regla de cancelación	Solo RECIBIDO. En otro estado el botón aparece apagado con el aviso 'Solo se cancela antes de empezar el lavado'
DetallePedidoScreen	Cabecera con la fecha y el chip de estado, StepperEstado vertical con las etapas, lista de líneas (servicio, cantidad × precio, subtotal), total y un recuadro de pago ('Pago pendiente, se paga al recoger' o 'Pagado')

Elemento	Debe cumplir
Casillero	Si el pedido está LISTO muestra el casillero asignado; si no, 'Se asignará al estar listo'
Confirmación	Cancelar pide confirmación con un AlertDialog. El pedido pasa a CANCELADO
Tiempo real	Si el administrador cambia el estado, la pantalla se actualiza sola

Resultado esperado:



5. Detalle y seguimiento
(Estudiante)

Verifica antes del commit: abres un pedido RECIBIDO y lo cancelas; abres uno EN_PROCESO y no te deja.

COMMIT 10 **feat(ui): agrega detalle del pedido y cancelacion del estudiante**

Paso 10 · Mis pedidos y Tarifas TÚ LO HACES

Objetivo: Listar los pedidos del estudiante en Activos e Historial y mostrar las tarifas vigentes.

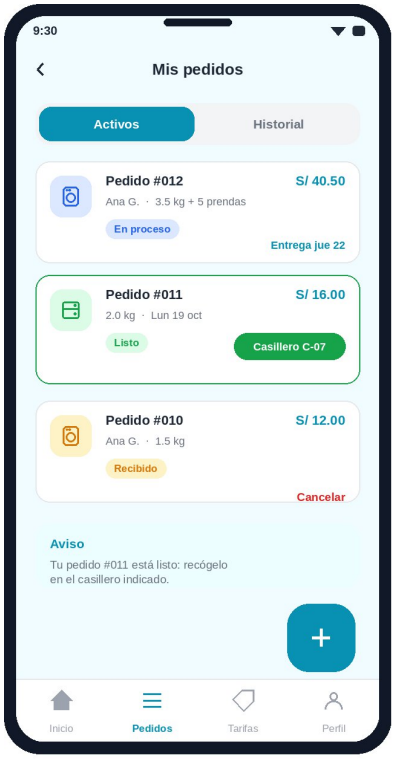
Archivos:

- viewmodel/MisPedidosViewModel.kt
- ui/estudiante/MisPedidosScreen.kt
- ui/estudiante/TarifasScreen.kt

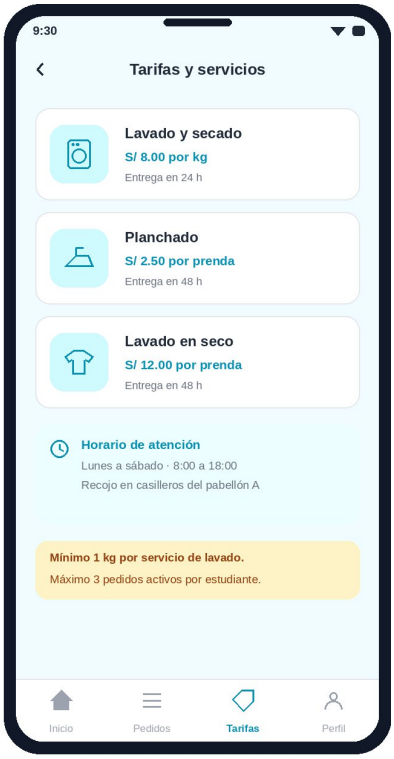
Contrato (lo que debe cumplir):

Elemento	Debe cumplir
MisPedidosViewModel	activos e historial: StateFlow. Activos = RECIBIDO, EN_PROCESO y LISTO; Historial = ENTREGADO y CANCELADO. Función cancelar(pedido) con la misma regla del Paso 9
MisPedidosScreen	Pestañas Activos / Historial, PedidoCard con EstadoPedidoChip, enlace 'Cancelar' donde corresponda, destaque verde con el casillero en los pedidos LISTO y FAB para crear un pedido
TarifasScreen	Lista de servicios ACTIVOS (observarServiciosActivos) con IconoServicio, nombre, precio por unidad ('S/ 8.00 por kg') y tiempo de entrega; tarjeta con el horario de atención y las reglas (mínimo 1 kg, máximo 3 pedidos activos)
Texto de la unidad	Una función de extensión Servicio.textoPrecio() convierte KG, PRENDA y UNIDAD en 'por kg', 'por prenda' y 'por unidad'

Resultado esperado:



6. Mis pedidos
(Estudiante)



7. Tarifas y servicios
(Estudiante)

Verifica antes del commit: un pedido entregado pasa a Historial; un servicio desactivado desde el administrador desaparece de Tarifas.

COMMIT 11 **feat(ui): agrega mis pedidos y tarifas**

Parte C · Flujo del Administrador

Paso 11 · Panel de la lavandería TÚ LO HACES

Objetivo: Crear el panel con contadores, ingresos y gráficos (reutilizados del laboratorio de Canvas).

Archivos:

- viewmodel/PanelViewModel.kt
- ui/administrador/PanelScreen.kt

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
PanelViewModel	pedidosHoy, enProceso, porRecoger: StateFlow<Int>; ingresosSemana: StateFlow<Double> (desde el lunes de esta semana); porEstado y porServicio: StateFlow<List<DatoGrafico>>
PanelScreen	3 contadores de color, tarjeta violeta de ingresos de la semana, GraficoDona (pedidos por estado), GraficoBarras (ingresos por servicio) y tarjeta de alerta con botón 'Ver' si hay pedidos LISTO esperando recojo
Inicio de semana	LocalDate.now().with(DayOfWeek.MONDAY).toString() (si hoy es domingo, ajusta con previousOrSame: TemporalAdjusters.previousOrSame(DayOfWeek.MONDAY))
Gráficos	GraficoDona y GraficoBarras ya vienen en ui/components/graficos y reciben List<DatoGrafico>. Tu trabajo es convertir los datos de Room a ese formato en el ViewModel

> **Pista**
> Para porEstado combina cuatro flujos con combine(...): observarCantidadPorEstado de cada EstadoPedido activo o entregado. Usa el color del estado (EstadoPedido.color) en cada DatoGrafico.

Resultado esperado:



2. Panel de la lavandería
(Administrador)

Verifica antes del commit: los contadores cambian cuando, como estudiante, creas un pedido; los ingresos coinciden con la suma de los pedidos pagados de la semana.

COMMIT 12 **feat(administrador): agrega panel con contadores y graficos**

Paso 12 · Bandeja y gestión del pedido TÚ LO HACES

Objetivo: Listar todos los pedidos y permitir al administrador avanzarlos de estado con las reglas de casillero y cobro.

Archivos:

- viewmodel/BandejaViewModel.kt
- viewmodel/GestionPedidoViewModel.kt
- ui/administrador/BandejaScreen.kt
- ui/administrador/GestionPedidoScreen.kt

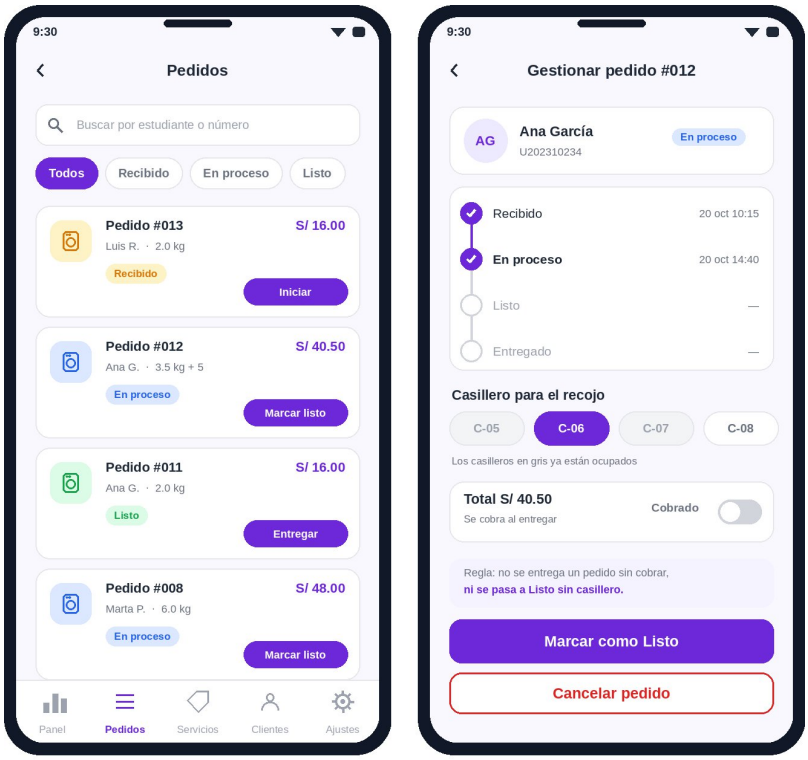
Contrato (lo que debe cumplir):

Elemento	Debe cumplir
BandejaViewModel	pedidos: StateFlow filtrado por el chip elegido (Todos, Recibido, En proceso, Listo) y por el texto del buscador (nombre del estudiante o número) usando combine
BandejaScreen	Buscador, chips de filtro y tarjetas con estudiante, detalle, total, estado y un botón de acción según el estado: Iniciar (RECIBIDO), Marcar listo

Elemento	Debe cumplir
	(EN_PROCESO) y Entregar (LISTO). Tocar la tarjeta abre la gestión
GestionPedidoViewModel	pedido, casillerosOcupados y el casillero elegido; funciones avanzar(), cancelar(), elegirCasillero(c) y marcarPagado(b)
Regla: avanzar	Solo al estado siguiente (EstadoPedido.siguiente()). Para pasar a LISTO exige un casillero elegido y libre; para pasar a ENTREGADO exige pagado = true. Si falla, se muestra el motivo
Regla: cancelar	Solo desde RECIBIDO o EN_PROCESO
GestionPedidoScreen	Datos del estudiante, StepperEstado vertical, selector de casillero con chips (los ocupados en gris y deshabilitados), tarjeta de total con el switch 'Cobrado', recordatorio de la regla y botones 'Marcar como ...' (siguiente estado) y 'Cancelar pedido'

> **Pista**
> Casilleros.todos genera C-01 a C-30. Los ocupados salen de observarCasillerosOcupados(). El casillero actual del propio pedido no cuenta como ocupado.

Resultado esperado:



3. Bandeja de pedidos (Administrador)

4. Gestionar pedido (Administrador)

Verifica antes del commit: no puedes pasar un pedido a Listo sin casillero ni entregarlo sin cobrar; el estudiante ve el cambio al instante.

COMMIT 13 **feat(administrador): agrega bandeja y gestion de pedidos con casilleros**

Paso 13 · Servicios, clientes, ajustes y cambio de perfil TÚ LO HACES

Objetivo: Completar la gestión de servicios y los ajustes del administrador, el perfil del estudiante y la lista de clientes, y cerrar el ciclo con el cambio de perfil.

Archivos:

- viewmodel/ServiciosViewModel.kt (completar TODO)
- ui/administrador/FormularioServicioScreen.kt
- viewmodel/ClientesViewModel.kt
- ui/administrador/ClientesScreen.kt
- ui/estudiante/PerfilScreen.kt
- ui/administrador/AjustesScreen.kt
- ui/navigation/NavLava.kt (completar rutas y grafo de cada perfil)

Contrato (lo que debe cumplir):

Elemento	Debe cumplir
ServiciosViewModel	Completa guardar(servicio) (nuevo o editado) y eliminar(servicio). Un servicio que ya está en algún pedido no se elimina (muestra un mensaje); solo se desactiva
FormularioServicioScreen	Nombre (obligatorio), se cobra por (Kilo, Prenda o Unidad), precio > 0, tiempo de entrega (24, 48 o 72 h), descripción, ícono y switch 'Servicio activo'. Sirve para crear y para editar
ClientesScreen	Buscador y tarjetas con avatar de iniciales, nombre, código, cantidad de pedidos y total gastado (ClienteResumen)
PerfilScreen (estudiante)	Datos del usuario y contadores de pedidos activos, pedidos totales y kilos lavados; switches de modo oscuro y avisos; botón 'Cambiar de perfil' y 'Cerrar sesión'
AjustesScreen (administrador)	Datos del usuario, 'Cambiar de perfil', switches de modo oscuro, notificaciones y 'Recibir pedidos', y 'Cerrar sesión'
Recibir pedidos	Si está apagado, el estudiante ve 'La lavandería no está recibiendo pedidos' y no puede crear uno
Cambiar de perfil	Guarda el otro perfil y navega a su grafo; limpia la pila de navegación (popUpTo)
Persistencia	Al cerrar y abrir la app, el modo oscuro y el perfil se conservan

Ajustes y perfil, vistas de referencia:

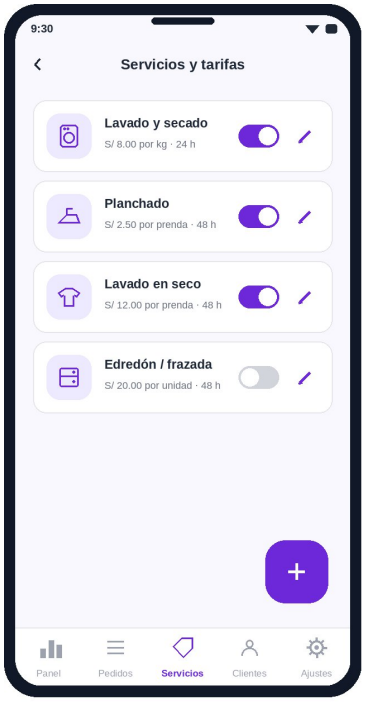


8. Perfil y preferencias
(Estudiante)

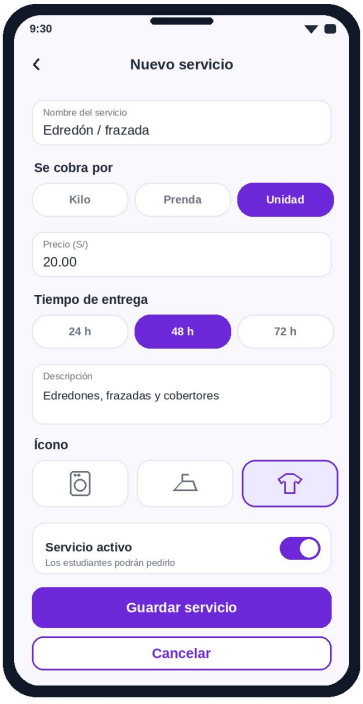


8. Ajustes (DataStore)
(Administrador)

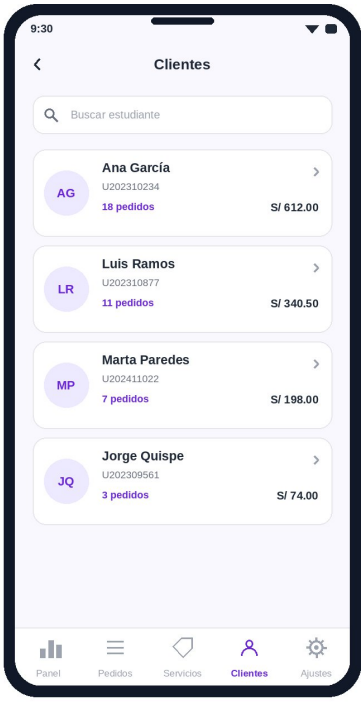
Resultado esperado:



5. Servicios y tarifas
(Administrador)



6. Nuevo / editar servicio
(Administrador)



7. Clientes
(Administrador)

Verifica antes del commit: desactivas un servicio y deja de aparecer al hacer un pedido; activas modo oscuro, cierras la app por completo y al abrir sigue oscuro.

COMMIT 14 **feat: agrega servicios, clientes, ajustes y cambio de perfil**

Paso 14 · Pruebas de aceptación y README

Marca cada prueba en el emulador. Todas deben pasar antes de cerrar la Fase 1.

Prueba	Resultado esperado	OK
Login con clave incorrecta	Mensaje de error, no entra	☐
Estudiante: nuevo pedido sin servicios	El botón Confirmar está apagado	☐
Estudiante: 3.5 kg de lavado + 5 prendas de planchado	Total S/ 40.50 y entrega a 2 días (saltando domingo)	☐
Estudiante: crear un cuarto pedido activo	Mensaje de límite y no se crea	☐
Estudiante: cancelar un pedido EN_PROCESO	No se permite cancelar	☐
Administrador: pasar a Listo sin casillero	Se rechaza con un mensaje	☐
Administrador: elegir un casillero ocupado	Aparece gris y no se puede elegir	☐
Administrador: entregar un pedido sin cobrar	Se rechaza; al marcar Cobrado sí se entrega	☐
Estudiante: ver el cambio de estado	El seguimiento se actualiza sin cerrar la pantalla	☐
Administrador: ingresos de la semana	Coinciden con la suma de los pedidos pagados	☐
Administrador: eliminar un servicio usado	No se elimina; se muestra un mensaje	☐
Cerrar y abrir la app	Los pedidos siguen ahí y el perfil y el modo oscuro se recuerdan	☐

Crea el README.md con: nombre, descripción, árbol de paquetes, el diagrama MVVM, 6 capturas (3 de cada perfil) y las respuestas de 'Para pensar'.

COMMIT 15 **docs: agrega README con estructura y capturas**
Fin de la Fase 1.

```
git add .
git commit -m "docs: agrega README con estructura y capturas"
git push origin main
```

Resumen de commits de la Fase 1

N°	Mensaje del commit	Contenido
1	chore: importa proyecto base LavaCampus	Proyecto base
2	feat(data): agrega entidades Usuario, Pedido y DetallePedido	entity/
3	feat(data): agrega DAOs y relaciones PedidoConDetalle y	dao/, relation/

N°	Mensaje del commit	Contenido
	DetalleConServicio	
4	feat(data): registra entidades y siembra datos iniciales en LavaDatabase	LavaDatabase, DatosIniciales, LavaApp
5	feat(data): implementa LavaRepositoryRoom con todas las operaciones	repository/
6	feat(data): agrega PreferenciasStore con DataStore	prefs/
7	feat(viewmodel): agrega SesionViewModel y login con selector de rol	SesionViewModel, LoginScreen
8	feat(ui): agrega inicio del estudiante con seguimiento del pedido	InicioViewModel + pantalla
9	feat(ui): agrega nuevo pedido con total y fecha de entrega	NuevoPedidoViewModel + pantalla
10	feat(ui): agrega detalle del pedido y cancelacion del estudiante	DetallePedidoViewModel + pantalla
11	feat(ui): agrega mis pedidos y tarifas	MisPedidosViewModel + 2 pantallas
12	feat(administrador): agrega panel con contadores y graficos	PanelViewModel + pantalla
13	feat(administrador): agrega bandeja y gestion de pedidos con casilleros	2 ViewModels + 2 pantallas
14	feat: agrega servicios, clientes, ajustes y cambio de perfil	Formulario, Clientes, Perfil, Ajustes, NavLava
15	docs: agrega README con estructura y capturas	README.md

FASE 2 · CON IA · Rama mejora-ia

Mejora profesional asistida por Gemini

Cuando la Fase 1 esté completa y subida a main, crea una rama nueva y usa Gemini en Android Studio para mejorar LavaCampus sin romper la arquitectura MVVM + Repository.

```
git checkout -b mejora-ia
```

Mejoras entre las que eliges (mínimo 3)

Mejora	Qué debe lograr	Capas que cambian
Gráfico de gasto mensual	El estudiante ve en su Perfil una línea con lo gastado en cada mes (reutiliza GraficoLineas del laboratorio de Canvas)	DAO + Repository + ViewModel + View
Repetir un pedido	Desde el historial, un botón 'Pedir de nuevo' carga las mismas líneas en Nuevo pedido con los precios actuales	ViewModel + View
Calificación del pedido	Al entregarse, el estudiante califica de 1 a 5 estrellas; el panel muestra el promedio	Entidad + DAO + Repository + ViewModel + View
Descuento por volumen	Si el lavado supera 5 kg se aplica 10 % de descuento visible en el resumen y guardado en el pedido	Entidad + Repository + ViewModel + View
Pedidos atrasados	La bandeja marca en rojo los pedidos cuya fecha de entrega ya pasó y no están entregados; el panel los cuenta	DAO + ViewModel + View
Recibo del pedido	Pantalla de recibo con el detalle y opción de compartirlo como texto	ViewModel + View

Condición obligatoria

Todo lo que genere la IA debe respetar MVVM: la View no usa DAO ni Repository; el ViewModel solo conoce la interface LavaRepository; si agregas una función al Repository, la declaras primero en la interface. Si cambias o agregas una entidad, sube la version de la base. Si Gemini rompe estas reglas, lo corriges y lo anotas en PROMPTS.md.

Ejemplo de prompt (puedes mejorarlo)

```
Tengo una app Kotlin con Jetpack Compose, Material 3, MVVM, Room y un
LavaRepository (interface). Agrega en la bandeja del administrador una
marca roja para los pedidos atrasados (fechaEntrega menor que hoy y
estado distinto de ENTREGADO y CANCELADO). La consulta va en PedidoDao,
la funcion en LavaRepository y su implementacion, y el valor se expone
como StateFlow en BandejaViewModel. La View solo lo muestra. No uses XML
ni librerias externas. Muestrame solo los archivos que cambian.
```

Commits mínimos de la Fase 2

COMMIT A	**ia: agrega pedidos atrasados en la bandeja**
COMMIT B	**ia: agrega descuento por volumen**
COMMIT C	**ia: agrega repetir un pedido**
COMMIT D	**docs: agrega PROMPTS.md** Documento con todos los prompts usados.

```
git push -u origin mejora-ia
```

Contenido de PROMPTS.md

```
# Prompts utilizados

## Prompt 1
**Texto enviado a Gemini:** ...
**Que genero la IA:** ...
**Que tuve que corregir yo:** ...
**Respeta MVVM?:** Si / No (explica)

## Reflexion (5 lineas)
- Que mejoro la IA y que no?
- Que error cometio?
- Que aprendi sobre MVVM, Room y reglas de negocio?
```